

Secure AWS CI/CD Pipeline

with Automated DevSecOps Security Gates

Project Guide & Engineering Reference

A factual technical reference for a real, deployed system: five enforced security scanners gating every deploy, Terraform-provisioned AWS infrastructure, and the complete production debugging record from a real AWS account.

Author: Raghav Anand • raghavanand033@gmail.com

AWS production endpoint: secure-cicd-dev-alb-907205721.ap-south-1.elb.amazonaws.com

Supplementary API demo: secure-aws-cicd-demo-api.vercel.app

Document status: reflects the current, live-deployed state of the project, including its real AWS production debugging history.

Table of Contents

1. Executive Summary	4
2. Problem Statement & Motivation	4
Advisory scanning vs. enforced scanning	5
3. System Architecture	5
3.1 High-level system flow	5
3.2 CI/CD pipeline sequence	5
3.3 Security-gate decision flow	6
3.4 AWS infrastructure topology	6
3.5 Threat / data-flow boundaries	6
4. Security Gates in Detail	7
4.1 SAST — Semgrep	7
4.2 Dependency / SCA — npm audit + OWASP Dependency-Check	8
4.3 Secret scanning — Gitleaks	8
4.4 IaC scanning — Checkov	8
4.5 Container scanning — Trivy	8
4.6 SBOM generation — Syft (non-blocking)	8
4.7 The fail-closed policy engine (security-gate.sh)	9
4.8 Configured thresholds (security/policy/security-policy.yaml)	9
5. AWS Infrastructure (Terraform)	9
5.1 IAM least-privilege design	10
5.2 KMS encryption scope	10
5.3 Networking — VPC, subnets, ALB	11
6. Secrets Management	11
6.1 The corrected Vercel secrets approach (the real fix made)	11
7. CI/CD Workflow	11
7.1 install	11
7.2 pre_build	12
7.3 build	12
7.4 post_build	12
7.5 A buildspec-specific YAML gotcha	12
7.6 The GitHub Actions mirror workflow	13

8. Monitoring & Observability	13
9. Application Security Controls	14
10. Production Deployment & Debugging Experience	15
10.1 Incident 8 in detail — the most instructive one	16
10.2 Incident 6 in detail — trusting CloudTrail over a vendor's own documented policy	17
10.3 What this demonstrates for the project's threat/reliability model	17
11. Validation & Testing	17
12. Limitations	18
13. Future Improvements	18
14. Local Setup & Reproduction Instructions	19
14.1 Prerequisites	19
14.2 GitHub setup	19
14.3 Terraform state backend (one-time, per AWS account)	19
14.4 Terraform deployment	20
14.5 Cost notes for a student/free-tier AWS account	20
14.6 Local development	20
14.7 terraform destroy cleanup	20
15. Research Component Summary	21
16. Live Demo Access	21
16.1 Demo walkthrough	22
16.2 Intentional-vulnerability demonstration (summary)	22
17. Appendix: Repository Structure Reference	23

1. Executive Summary

This project is a small Node.js/Express REST API wrapped in a full DevSecOps CI/CD pipeline in which security scanning is a **blocking, automated gate** rather than an advisory report. On every build, five independent scanner categories run — static application security testing (SAST), dependency/software-composition analysis (SCA), secret scanning, infrastructure-as-code (IaC) scanning, and container image scanning. A policy engine (`security-gate.sh`) reads each scanner's normalized JSON report, evaluates it against configurable severity thresholds in `security/policy/security-policy.yaml`, and fails closed: a missing report, a malformed report, or a scanner crash is treated identically to an actual policy violation. There is no manual “approve to deploy” step anywhere in the pipeline — the gate's pass/fail decision is the only thing standing between a commit and a production deployment.

The application layer (`app/`) exists to give the pipeline something real to build, scan, and deploy: JWT-based authentication, bcrypt password hashing, Helmet security headers, rate limiting, input validation, and structured, redacted logging. The infrastructure layer (`terraform/`, eight modules) provisions a VPC with public/private subnets, an Application Load Balancer, an ECS Fargate service, an ECR registry, IAM roles scoped per actor, KMS encryption, and CloudWatch observability. The pipeline layer (`ci/buildspec.yaml`, `.github/workflows/`) wires the gate into an actual AWS CodePipeline / CodeBuild / ECR / ECS Fargate deployment.

Current live status. This is not a design exercise sitting only in source control. The full pipeline is deployed and running in a real AWS account (region `ap-south-1`), and a supplementary public copy of just the API runs on Vercel for quick, AWS-free poking. Both are described in detail in Section 16 (Live Demo Access). Getting the AWS deployment to a genuinely healthy state surfaced eight real, documented production incidents — region-specific assumptions, undocumented IAM permission requirements, a stricter CodeBuild YAML parser, and a distroless-image health-check bug that required log-level evidence, not just the error text, to root-cause. That full debugging record is reproduced faithfully in Section 10, because it is some of the most engineering-relevant content in this guide.

Deployment	URL	Exercises the security gate?
AWS production (CodePipeline → CodeBuild → ECS Fargate)	<code>secure-cicd-dev-alb-907205721.ap-south-1.elb.amazonaws.com</code>	Yes — every deploy
Vercel API demo (supplementary)	<code>secure-aws-cicd-demo-api.vercel.app</code>	No — Vercel's build does not run the scanners or the gate

Table 1. The two currently live, verified-reachable endpoints this project maintains.

2. Problem Statement & Motivation

Security scanning is frequently bolted onto CI/CD pipelines as an **advisory**, non-blocking step: a scan runs, a report is generated, a dashboard gets a new red badge — and the deployment proceeds regardless of the result. This decouples *detection* from *prevention*: the scan can be perfectly accurate and still have zero effect on what actually ships, because nothing in the pipeline's control flow depends on its result. The specific, mechanical question this project treats as a first-class engineering problem is not “which scanner to run,” but *how to make enforcement itself automated, fail-closed, and resistant to silent bypass* — as opposed to adding a manual approval step and hoping someone reads the report before clicking approve.

The gate (`security/scripts/security-gate.sh`) is therefore treated as a piece of software with its own specification and its own fixture-based test suite (`tests/security/run-tests.sh`), not as pipeline glue that happens to call some scanners. Section 15 summarizes the research framing this motivates in more depth; the full academic treatment is the separate IEEE-format paper, `docs/research-paper-ieee.pdf`.

Advisory scanning vs. enforced scanning

Property	Advisory scanning (common pattern)	Enforced gate (this project)
Effect of a finding	Report generated; deploy proceeds regardless	Deploy is blocked (CodeBuild phase fails, CodePipeline never reaches Deploy)
Missing/crashed scanner	Silently skipped or ignored	Fails closed — treated as a violation
Bypass path	A human can ignore the report	No manual approval step exists to route around
Policy location	Often implicit or scattered	Single versioned file, <code>security-policy.yaml</code>

Table 2. The distinction this project's design is built around, per `README` and `security-model.md`.

3. System Architecture

The end-to-end flow, described in full in `docs/architecture.md` as Mermaid diagrams, is reproduced here as prose and text diagrams (Mermaid source does not render in a PDF). Everything downstream of the Security Gate only runs if the gate passes; there is no manual approval action anywhere in CodePipeline.

3.1 High-level system flow

```
Developer --git push--> GitHub --(CodeStar Connection webhook)--> CodePipeline
```

```
|
```

```
v
```

```
CodeBuild
```

```
(scans + gate decision)
```

```
|
```

```
+-----+-----+
```

```
|
```

```
|
```

```
Gate PASS Gate FAIL
```

```
|
```

```
|
```

```
v
```

```
v
```

```
v
```

```
v
```

```
ECS Fargate Service <--- ALB <--- End User (HTTPS)
```

CloudWatch Logs & Alarms receive log/metric output from CodeBuild, CodePipeline, and the running ECS task. Secrets Manager / SSM resolve secrets and config into the ECS task at container startup. A shared KMS CMK encrypts ECR, Secrets Manager, CloudWatch Logs, and the pipeline's S3 artifact bucket.

3.2 CI/CD pipeline sequence

The Build stage, run by CodeBuild against `ci/buildspec.yml`, proceeds strictly in this order:

1. Developer pushes to main on GitHub
2. GitHub notifies CodePipeline's Source stage via the CodeStar Connection
3. CodePipeline starts the Build stage -> CodeBuild runs `ci/buildspec.yml`
4. CodeBuild: install dependencies, lint, unit tests

```

5. CodeBuild: docker build
6. CodeBuild: run SAST / SCA / secret / IaC / container scans (each writes a normalized
  JSON report; a non-zero scan exit code alone does NOT fail the build)
7. CodeBuild: security-gate.sh evaluates every report against the policy
  Gate PASS -> docker push to ECR (digest-pinned) -> imagedefinitions.json emitted
  -> Build stage SUCCEEDED -> CodePipeline Deploy stage updates the ECS
  service -> new revision running behind the ALB
  Gate FAIL -> security-gate.sh exits non-zero -> the build command fails -> CodeBuild
  project FAILED -> CodePipeline stops before Deploy -> ECS is never touched

```

3.3 Security-gate decision flow

`security-gate.sh` itself follows this logic for every run:

```

security-gate.sh starts
-> Load security-policy.yaml
invalid YAML / missing required section --> exit 3 ("gate cannot run", hard abort)
valid --> for each category (SAST, Dependencies, Secrets, IaC, Container):
  report file exists, valid JSON, status == "completed"?
  NO --> category = FAIL (fail-closed)
  YES --> findings within policy thresholds?
  NO --> category = FAIL (threshold violated)
  YES --> category = PASS
aggregate all category results
any category FAIL? --> OVERALL RESULT: FAIL, exit 1, deployment blocked
none FAIL? --> OVERALL RESULT: PASS, exit 0, deployment proceeds

```

3.4 AWS infrastructure topology

```

VPC (10.20.0.0/16)
Public Subnet AZ-1: ALB, NAT Gateway Public Subnet AZ-2: ALB
Private Subnet AZ-1: ECS Fargate Task Private Subnet AZ-2: ECS Fargate Task
Internet Gateway <-> both public subnets

Internet -> Internet Gateway -> ALB (public subnets) -> ECS Fargate tasks (private subnets)
ECS tasks -> outbound only, via the single NAT Gateway -> Internet Gateway
ECS tasks -> pull image from ECR | resolve secrets from Secrets Manager/SSM | ship logs to
CloudWatch

```

- ECS tasks run in **private subnets only**, with no public IP; the ALB in the public subnets is the sole ingress path.
- The ECS tasks' security group accepts inbound traffic **only** from the ALB's security group, on the container port.
- Outbound access from private subnets goes through a NAT gateway — there is no direct route from a private subnet to the internet gateway.

3.5 Threat / data-flow boundaries

`architecture.md` also defines a numbered threat/data-flow diagram that maps directly onto the STRIDE analysis in Section §4 of the threat model: flows (1)/(2) are the Attacker → Internet → ALB → ECS path (rate limiting, input validation, injected headers); flow (3) is the ECS task reading its JWT secret from Secrets Manager at startup; flows (4)-(6) are GitHub source → CodeBuild → the gate → ECR (why the gate exists at all); and flow (7) is the ECS agent's IAM-scoped pull from ECR (immutable tags, scan-on-push, least-privilege IAM).

Component	Responsibility
GitHub	Source of truth for application code, IaC, and pipeline config
CodeStar Connection	Authenticated, tokenless link from CodePipeline to GitHub
CodePipeline	Orchestrates Source → Build → Deploy, no manual gates
CodeBuild	Runs ci/buildspec.yml: build, test, scan, gate, push
security-gate.sh	Policy engine — the actual pass/fail decision maker
ECR	Immutable, scan-on-push image registry
ECS Fargate	Runs the container, no server management
ALB	Load-balances traffic to ECS tasks (plain HTTP in the current demo deployment)
Secrets Manager / SSM	Runtime secret/config injection, never baked into the image
CloudWatch	Logs, metrics, alarms, SNS notifications
KMS	Encryption at rest for ECR, Secrets Manager, CloudWatch Logs, S3

Table 3. Component responsibilities, verbatim from docs/architecture.md.

4. Security Gates in Detail

Five independent scanner categories run on every build, each with its own wrapper script under `security/scripts/` and its own normalized JSON report format under `security/reports/<category>/. A scanner that cannot find its tool writes a report with status: "tool_missing" and exits 1 — it does not silently skip itself — and a scanner that errors mid-run writes status: "error" and exits 2. The gate treats both of these, and a flat-out missing or corrupt report file, as a violation, identically to an actual policy threshold breach.`

```
{
  "scanner": "trivy",
  "status": "completed | tool_missing | error",
  "generated_at": "2026-01-01T00:00:00Z",
  "findings": { "critical": 0, "high": 0, "medium": 0, "low": 0 },
  "details": [ /* up to 50 individual findings */ ]
}
```

Figure. The normalized report shape every scanner wrapper produces (security-model.md).

4.1 SAST — Semgrep

Checks: static code patterns — injection, unsafe crypto, insecure deserialization, and similar known-bad patterns. Wired in via `bash security/scripts/run-sast.sh app/src` in the CodeBuild build phase; its own exit code is captured but not itself fatal — only the gate's evaluation of its report is. Installed in CodeBuild via `pip install semgrep` in the `install` phase (best-effort; a failed install falls through to a `tool_missing` report, which the fail-closed gate then blocks on).

4.2 Dependency / SCA — npm audit + OWASP Dependency-Check

Checks: known-vulnerable dependencies in the application's npm dependency tree. `npm audit` requires no separate install (it ships with npm); OWASP Dependency-Check is the optional, deeper SCA layer, installed locally via Homebrew or the CLI zip per `troubleshooting.md`. Wired in via `bash security/scripts/run-dependency-scan.sh` app.

4.3 Secret scanning — Gitleaks

Checks: committed secrets, including in git history, not just the working tree. This is the one category with **zero-tolerance** (`secrets.block_on_any_finding: true`) rather than a severity threshold — a single leaked credential is unconditionally unacceptable regardless of how the scanner classified its severity. Wired in via `bash security/scripts/run-secret-scan.sh`; installed in CodeBuild by downloading the latest Linux release binary directly from GitHub (Gitleaks does not ship a convenience install script the way Trivy does).

4.4 IaC scanning — Checkov

Checks: misconfigured infrastructure as code — public S3 buckets, open security groups, missing encryption, overly broad IAM, and similar. Scans the entire `terraform/` tree. Wired in via `bash security/scripts/run-iac-scan.sh` `terraform`; installed via `pip install checkov`. A small number of specific, justified findings are suppressed inline in `terraform/main.tf` with `checkov:skip` comments (e.g. cross-region replication and event notifications on the `artifact/access-log` buckets), each with a one-line rationale rather than a blanket exclusion.

4.5 Container scanning — Trivy

Checks: known-vulnerable OS packages and language dependencies baked into the built container image. Wired in via `bash security/scripts/run-container-scan.sh "$LOCAL_IMAGE"` against the image CodeBuild just built. This is a second, independent check on top of ECR's own scan-on-push configuration (`terraform/modules/ecr, image_scanning_configuration`). Installed via the upstream Trivy install script.

4.6 SBOM generation — Syft (non-blocking)

`ci/scripts/generate-sbom.sh` runs Syft (when available) against the built image and emits both CycloneDX and SPDX SBOMs. This step is explicitly best-effort (`|| true` in the builds spec) and **currently informational, not a gate input** — wiring SBOM diffing into the gate is listed as future work in Section 13.

4.7 The fail-closed policy engine (security-gate.sh)

The gate's decision procedure, precisely: (1) parse `security-policy.yaml` — if it is missing, unparseable, or missing a required top-level section, abort hard with **exit 3** (the gate itself is broken, so nothing should proceed on the assumption it passed); (2) for each enabled category, load its report — a missing file, invalid JSON, or a status other than "completed" counts as a fail-closed violation unless the policy explicitly opts that category into fail-open behavior (no category currently does); otherwise compare `findings.<severity>` against `<category>.block_on_severity.<severity>`; (3) secrets get the zero-tolerance rule described above instead of a severity threshold; (4) print a `SECURITY GATE RESULT` table and a machine-readable summary, then **exit 0** (PASS) or **1** (FAIL). Enforcement happens purely because `ci/buildspec.yml` calls `security-gate.sh` as an ordinary shell command in the build phase: a non-zero exit fails that phase, which fails the CodeBuild project, which fails the CodePipeline Build stage, which means the Deploy stage — the only action that touches the running ECS service — never executes.

This is validated by `tests/security/run-tests.sh`, whose 7 fixture scenarios (all currently passing, see Section 11) cover: passing when all five categories are within policy; failing on a threshold violation; failing on any secret finding; failing closed on a missing report; failing closed on a malformed (non-JSON) report; failing closed on a scanner-crashed status; and failing closed on a tool-not-installed status.

4.8 Configured thresholds (security/policy/security-policy.yaml)

Semantics: a threshold of 0 means any finding at or above that severity blocks the deployment; a threshold of $N > 0$ tolerates up to N findings at that severity (useful for MEDIUM/LOW noise while a backlog is burned down); -1 means unlimited/not enforced. CRITICAL and HIGH are hard-blocked (0) across every category by default.

Category	Critical	High	Medium	Low
SAST (Semgrep)	0	0	10	unlimited
Dependencies (npm audit / OWASP DC)	0			
Secrets (Gitleaks)	zero-tolerance — any finding blocks (not severity-based)			
IaC (Checkov)	0	0	15	unlimited
Container (Trivy)	0	0	20	unlimited

Table 4. Exact thresholds from `security/policy/security-policy.yaml` (version 1, `policy_name: default-devsecops-gate`).

Global gate behavior flags, also from the policy file: `gate.fail_closed_on_missing_report: true` and `gate.fail_closed_on_scanner_crash: true`; summaries are written to `security/reports/summary`.

5. AWS Infrastructure (Terraform)

Infrastructure is modular Terraform (AWS provider $\rightarrow$ 5.0), eight modules under `terraform/modules/`, wired together by the root module (`terraform/main.tf`) in dependency order: `security` $\rightarrow$ `networking` $\rightarrow$ `ecr` $\rightarrow$ (root artifact/access-log S3 buckets) $\rightarrow$ `iam` $\rightarrow$ `codebuild` $\rightarrow$ `ecs` $\rightarrow$ `cloudwatch` $\rightarrow$ `codepipeline`. A concrete environment (`terraform/environments/dev`) invokes the root module with real variable values.

Module	What it provisions
security	The shared KMS CMK and its alias; the JWT signing secret (random_password, AWS Secrets Manager secret + version); SSM Parameter Store entries for log level and rate-limit configuration.
networking	VPC, Internet Gateway, public and private subnets across the resolved availability zones, a single NAT Gateway + EIP, public/private route tables and associations, VPC Flow Logs (with their own CloudWatch log group and IAM role), the default security group, and dedicated security groups for the ALB, the ECS tasks, and CodeBuild.
ecr	The ECR repository (scan-on-push, immutable tags), its lifecycle policy (caps retained image history), and its repository policy (account-scoped).
iam	Four purpose-built IAM roles and their policies: CodePipeline, CodeBuild, the ECS task execution role, and the ECS task role — one role per actor, least privilege.
codebuild	The CodeBuild project, preconfigured to run ci/buildspec.yml from the repo root, in VPC mode (private subnets, its own security group), with privileged mode enabled (required for docker build, not itself an IAM privilege).
ecs	The ECS cluster, its own CloudWatch app log group, the Application Load Balancer, its target group and HTTP listener, the ECS task definition (container port, health check, secrets/SSM references), and the ECS Fargate service.
cloudwatch	CodeBuild and CodePipeline log groups, the SNS alerts topic and its email subscription, and three CloudWatch metric alarms: ECS running-task-count-low, ALB 5xx-high, and ALB latency-high.
codepipeline	The CodePipeline itself: Source (CodeStar Connection to GitHub) → Build (the CodeBuild project above) → Deploy (updates the named ECS cluster/service) — no manual approval stage.

Table 5. The eight Terraform modules and their concrete resources, per terraform/main.tf and each module's main.tf.

5.1 IAM least-privilege design

Every role in terraform/modules/iam is single-purpose (CodePipeline, CodeBuild, ECS task execution, ECS task), with every policy statement justified by an inline comment, and no wildcard Resource: "*" except where AWS mandates it (documented inline where that occurs, e.g. ecr:GetAuthorizationToken, cloudwatch:PutMetricData). This design was directly tested against real AWS behavior during deployment: Incident 3 and Incident 6 in Section 10 are both cases where an IAM permission AWS's own documentation does not mention had to be discovered and added.

5.2 KMS encryption scope

A single shared KMS CMK, provisioned in the security module, encrypts: the ECR repository, the Secrets Manager secret holding the JWT signing key, CloudWatch Logs, and the pipeline's S3 artifact bucket. The access-log destination bucket is the one exception — ALB access-log delivery only supports SSE-S3 (AES256), not SSE-KMS, which is an AWS ALB limitation rather than a relaxed control; the bucket is still always encrypted at rest, just with a different mechanism (documented inline in terraform/main.tf next to the corresponding Checkov skip).

5.3 Networking — VPC, subnets, ALB

The VPC spans two availability zones with a public and a private subnet in each. The public subnets host the ALB and the single NAT Gateway; the private subnets host the ECS Fargate tasks, which have no public IP. Availability zones are resolved dynamically (`data.aws_availability_zones.available`, sliced to the first two) rather than hardcoded, specifically because a hardcoded AZ list is not portable across AWS regions — this exact issue was Incident 1 in the production debugging log (Section 10). The ECS tasks' security group accepts inbound traffic only from the ALB's security group on the container port; the ALB is the sole ingress path. VPC Flow Logs are enabled and shipped to their own CloudWatch log group.

6. Secrets Management

On AWS, the JWT signing secret is generated by Terraform (`random_password`, never hardcoded anywhere in source), stored in AWS Secrets Manager, and resolved by the ECS task at container startup — it is never baked into the image or written to disk. Non-secret runtime configuration (log level, rate-limit thresholds) is passed via SSM Parameter Store. The ECS task execution role is scoped to read exactly the JWT secret's ARN and the two SSM parameter ARNs (`terraform/modules/iam`), not a broader Secrets Manager/SSM grant.

6.1 The corrected Vercel secrets approach (the real fix made)

The Vercel supplementary demo originally had `JWT_SECRET` committed in `app/vercel.json` as a labeled placeholder value. This was identified and fixed: even a clearly labeled placeholder JWT signing secret becomes the deployment's *real* signing key the moment Vercel builds it, so it was removed from `app/vercel.json` entirely and moved to a project-level, **encrypted** Vercel environment variable instead, set via `vercel env add JWT_SECRET production` (or the Vercel dashboard) with a freshly, independently generated random value that is never committed to source control. This secret is never reused for the AWS deployment, which gets its own independently generated JWT secret from Secrets Manager — the two deployments intentionally never share a signing key, so a compromise of one does not compromise the other.

```
# app/vercel.json now sets only NON-secret runtime env vars:
# JWT_ISSUER, rate-limit thresholds, log level
# JWT_SECRET is deliberately absent from this file.

vercel env add JWT_SECRET production
# paste a freshly generated random value -- never the old committed placeholder
```

Figure. The corrected pattern, from docs/deployment.md section 11.

This is a concrete, load-bearing example of the project's own security principles being applied to its own supplementary deployment, not just to the application code the gate scans.

7. CI/CD Workflow

`ci/buildspec.yml` is the exact, real CodeBuild pipeline definition — the only place in this pipeline where the security gate is enforced. It has four phases:

7.1 install

- Print tool versions (node, npm, docker).
- `cd app && npm ci` — installs application dependencies from the locked dependency tree.

- Best-effort install of the security scanners: Semgrep and Checkov via `pip install`; Gitleaks, Trivy, and Syft via direct download/install scripts. Each failure is caught and logged as a warning rather than failing the build immediately — the corresponding scan step will report `tool_missing` later, and the fail-closed gate blocks on that.
- Install `yq` (required by `security-gate.sh` to parse the policy YAML) — this one is explicitly called out as not best-effort, since a missing `yq` makes the gate hard-abort (exit 3) rather than just report one category as missing.

7.2 pre_build

- `npm run lint`
- `npm test` (unit tests)
- ECR login (`ci/scripts/ecr-login.sh`)
- Export `LOCAL_IMAGE` and `IMAGE_TAG` (the latter from `CODEBUILD_RESOLVED_SOURCE_VERSION`).

7.3 build

- `docker build` the application image.
- Run all five scanners in sequence (SAST, dependency, secret, IaC, container), each with its exit code captured into its own variable (`SAST_EXIT`, etc.) but not itself fatal.
- Best-effort, non-blocking SBOM generation via `ci/scripts/generate-sbom.sh`.
- Run `security-gate.sh` — this is the actual pass/fail decision, made from the normalized reports, not from the individual scan exit codes captured above.
- Only if the gate passes: push the built image to ECR, digest-resolved, via `ci/scripts/push-image.sh`.

7.4 post_build

- Print the gate's human-readable summary (`security/reports/summary/security-gate-summary.txt`) to the CodeBuild log for visibility, regardless of pass/fail.

Only `imagedefinitions.json` is emitted as the pipeline artifact — that is the one file CodePipeline's ECS deploy action consumes; the full scan reports and gate summary are captured in the CodeBuild CloudWatch log group instead of a second artifact, to keep this buildspec compatible with the single-output-artifact CodePipeline Build stage.

7.5 A buildspec-specific YAML gotcha

Every buildspec command containing a colon is wrapped in single quotes. AWS CodeBuild's buildspec YAML parser is stricter than most YAML libraries, including the one GitHub Actions uses: an unquoted `:` inside a plain scalar is parsed as the start of a nested mapping key, which fails with “found subkeys instead.” Wrapping the whole command in single quotes (doubling any literal single quotes inside it) avoids this, and this is exactly what Incident 4 in Section 10 surfaced on a real CodeBuild run — it did not show up in the mirrored GitHub Actions workflow at all.

7.6 The GitHub Actions mirror workflow

`.github/workflows/security-gate.yml` mirrors the same scan-then-gate sequence CodeBuild runs, for PR-time enforcement — but it is **validation only**, not a deployment path. It does not push images, update ECS, or touch any AWS deployment resource; its purpose is to give a required, gate-enforced GitHub status check (recommended as a branch-protection rule for `main` in `deployment.md`) so a pull request can be blocked from merging on the same gate logic, independent of and before any CodeBuild run. `.github/workflows/terraform-validate.yml` similarly validates Terraform (including a Checkov pass) on every change to `terraform/`.

8. Monitoring & Observability

Provisioned by the `cloudwatch Terraform` module. CloudWatch log groups exist for the application (`/ecs/<project>-<environment>-app`, provisioned in the `ecs` module), CodeBuild, and CodePipeline. Three metric alarms notify a single SNS topic (`aws_sns_topic.alerts`), with an email subscription configurable via the `alarm_notification_email` Terraform variable:

- **ECS running-task-count-low** — alarms if the number of running tasks drops below the expected count.
- **ALB 5xx-high** — alarms on an elevated ALB-reported 5xx error count.
- **ALB latency-high** — alarms on elevated p95 request latency at the ALB.

Application logs are structured (`pino`) and automatically redact `password/token/secret` fields by pattern (`app/src/utils/logger.js`); the centralized error handler never returns stack traces or raw error objects to clients. VPC Flow Logs (provisioned in the `networking` module) capture network-level traffic to their own log group. Notably, CloudWatch log evidence — not just an error message — was the decisive tool in root-causing Incident 8 in Section 10.

9. Application Security Controls

Layer	Controls	Where
Source code	Input validation, JSON output encoding, no eval/dynamic require	app/src
Authentication	bcrypt password hashing (cost 12), JWT with issuer claim and expiry, constant-time-shaped login response to resist user enumeration	app/src/controllers/authController.js
Transport	Helmet secure headers (CSP, HSTS, X-Frame-Options, etc.), rate limiting	app/src/app.js
Secrets	Never hardcoded; generated by Terraform, stored in Secrets Manager, resolved at container startup	terraform/modules/security, terraform/modules/ecs
Container	Multi-stage build, non-root user, minimal/distroless runtime base, readonlyRootFilesystem, health check, no unnecessary packages	app/Dockerfile, terraform/modules/ecs/main.tf
Registry	Immutable tags, scan-on-push, KMS encryption at rest, account-scoped repository policy	terraform/modules/ecr
Network	Private subnets for compute, security groups scoped to minimum required source/port, NAT for controlled egress only	terraform/modules/networking
IAM	One role per actor, every statement commented, no wildcard Resource except where AWS mandates it	terraform/modules/iam
Pipeline	Automated security gate enforced in code, fail-closed on any scanner or policy failure	security/scripts/security-gate.sh, ci/buildspec.yml
Observability	Structured, redacted logs; CloudWatch alarms on task health, error rate, latency	app/src/utis/logger.js, terraform/modules/cloudwatch

Table 6. Defense-in-depth layers, verbatim from docs/security-model.md section 1.

The Docker base image was chosen after comparing measured Trivy CVE counts across `node:20-alpine`, `node:22-alpine`, `node:22-slim`, and `distroless` (`app/Dockerfile`): the build stage uses `node:22-alpine`, and the runtime image uses `gcr.io/distroless/nodejs22-debian12:nonroot` as the smallest measured CVE surface among the options actually scanned. This same `distroless` choice is precisely what produced Incident 8 in Section 10 — a real example of a security-motivated decision introducing an operational correctness bug that had to be found and fixed on its own terms.

10. Production Deployment & Debugging Experience

This section is a factual record of every real failure encountered while deploying this project's Terraform-managed infrastructure and driving its CodePipeline to a healthy, publicly reachable ECS Fargate service in a real AWS account (region ap-south-1). It exists for two reasons: **engineering honesty** — a project that claims to be production-tested should be able to show exactly what production disagreed with, not just a clean final terraform apply output; and **research value** — several of these incidents are not obvious from reading AWS's own documentation and were only found by going past it (CloudTrail event correlation, docker inspect on a base image, reading application logs against ALB vs. container-internal traffic). Every root cause below was confirmed against real AWS API output before a fix was written; none is inferred or assumed. This is the complete list of deploy-blocking issues encountered getting from terraform apply to a healthy, ALB-served ECS Fargate task — not a curated highlight reel.

#	Symptom	Root cause	Layer	Found via
1	terraform apply failed: InvalidParameterValue on subnet AZs	Hardcoded us-east-1a/1b AZs don't exist in ap-south-1	Terraform/netw orking	Direct apply error output
2	4 CloudWatch log groups failed to create: AccessDeniedException	KMS key policy granted the account root but not the logs.<region>.amazonaws.com service principal	Terraform/KMS	Direct apply error output
3	CodeBuild: Not authorized to perform DescribeSecurityGroups	CodeBuild's VPC mode needs explicit EC2 ENI-management IAM permissions; not implied by any other role	IAM	CodeBuild build log
4	CodeBuild: YAML_FILE_ERROR on a builds spec that parsed fine in GitHub Actions	AWS CodeBuild's builds spec YAML parser is stricter than GitHub Actions'; an unquoted colon in a plain scalar is read as a nested mapping key	CI config	CodeBuild DOWNLOAD_S OURCE/parse failure
5	CodeBuild: yq: command not found	security-gate.sh requires yq; GitHub Actions runners ship it, the CodeBuild managed image does not	CI config	CodeBuild build log
6	CodePipeline Deploy stage: generic PermissionError, 3 consecutive failures despite matching AWS's own documented reference IAM policy	CodePipeline's ECS deploy action also calls ecs:TagResource when registering a new task definition revision — not present in AWS's own published reference policy	IAM	aws cloudtrail lookup-events, not the CodePipeline error message
7	ECS deployment circuit breaker tripped: tasks failed to start	The task definition's image referenced a real digest, but a later Terraform-only apply (fixing #8) re-registered the task definition using the module's bootstrap default image tag (:initial), which was never pushed	Terraform/ECS	aws ecs describe-services events: CannotPullContainerError
8	ECS container marked UNHEALTHY and killed after ~7 minutes, despite the app answering the ALB's own health checks the entire time	The container-level healthCheck invoked a bare "node"; exec-form CMD/HEALTHCHECK resolves argv[0] via \$PATH, not the image's ENTRYPOINT, and the distroless image's \$PATH excludes /nodejs/bin	Container image	Application logs showed zero requests from the internal health-check process; confirmed via docker inspect

Table 7. Incident summary, verbatim from docs/production-incidents.md.

10.1 Incident 8 in detail — the most instructive one

This is worth walking through because the failure signature is misleading by default, and the fix requires evidence a log line will never show you directly.

Symptom

`aws ecs describe-services` reported the deployment's `rolloutState` as `FAILED` with reason “ECS deployment circuit breaker: tasks failed to start,” and the ECS service event log showed the task failing container health checks and the service stopping the running task.

The misleading part

The ALB's own external health check (`ELB-HealthChecker/2.0`) was hitting `/health` on the task's private IP every 15 seconds and getting `200 OK` the entire time — for over 7 minutes, right up until the task was killed. If you only looked at the ALB target group's health state, or `curl'd` the load balancer yourself, the service looked completely healthy.

The actual investigation

Pulling the task's CloudWatch log stream directly (`aws logs get-log-events`) and grepping every `/health` request by its user-agent showed that **100% of them were `ELB-HealthChecker/2.0`** — not one request in the log ever came from the container's own internal health-check process (which, if it were reaching the app at all, would show up as a plain `Node.js http.get()` client with no special user agent). That absence is the actual signal: ECS's container-level `healthCheck` was never reaching the application process at all.

Root cause

The image is `gcr.io/distroless/nodejs22-debian12:nonroot`. Verified directly:

```
$ docker inspect gcr.io/distroless/nodejs22-debian12:nonroot --format '{{json .Config.Entrypoint}}'
["/nodejs/bin/node"]
$ docker inspect gcr.io/distroless/nodejs22-debian12:nonroot --format '{{json .Config.Env}}'
["PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin","SSL_CERT_FILE=/etc/ssl/certs/ca-certificates.crt"]
```

The image's own `ENTRYPOINT` is the absolute path to `node`, so the container's main process starts fine — `ENTRYPOINT` doesn't need a `$PATH` lookup. But `HEALTHCHECK/ECS's` `healthCheck.command` is an independent `exec()` that Docker/ECS resolves via `$PATH` lookup exactly like a shell would, and this image's `$PATH` is the standard system directories only — it does not include `/nodejs/bin`. Every single health check invocation failed with (effectively) “executable file not found in `$PATH`” before it ever made a TCP connection, exactly consistent with the log evidence.

Fix

```
HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 \
  CMD ["/nodejs/bin/node", "-e", "..."]

healthCheck = {
  command = ["CMD", "/nodejs/bin/node", "-e", "..."]
  ...
}
```

Why this generalizes

Any distroless (or otherwise shell-less, minimal-\$PATH) base image will hit this exact failure mode for any exec-form health check that assumes the entrypoint binary is on \$PATH — it is a property of how Docker/ECS execute CMD-form health checks, not something specific to Node.js or this one image. The general lesson: an exec-form health check must use the same absolute path as the image's own ENTRYPOINT, verified with `docker inspect`, never assumed.

10.2 Incident 6 in detail — trusting CloudTrail over a vendor's own documented policy

AWS publishes a documented “reference” IAM policy for CodePipeline's ECS deploy action. Two consecutive attempts matched it exactly — once with a narrowly scoped custom policy, once by copying AWS's own reference policy verbatim — and both still failed with the same generic “not sufficient permissions to access ECS” message, which CodePipeline does not expand with the specific denied action.

`aws cloudtrail lookup-events` (filtering for `AccessDenied` around the failed execution's timestamp) surfaced the actual denial:

```
User: arn:aws:sts::<account>:assumed-role/<name>-codepipeline-role/<session>
is not authorized to perform: ecs:TagResource
on resource: arn:aws:ecs:<region>:<account>:task-definition/<name>-app:*
because no identity-based policy allows the ecs:TagResource action
```

CodePipeline's ECS deploy action propagates the task definition family's tags onto every new revision it registers — an implementation detail not present in AWS's own published reference policy for this feature. The fix was a single scoped statement granting `ecs:TagResource` on the task definition family ARN. **Lesson:** for an opaque IAM `AccessDenied` without a specific action named in the error, CloudTrail's raw event log is more reliable than a cloud provider's own “complete” reference policy.

10.3 What this demonstrates for the project's threat/reliability model

None of these eight incidents were security vulnerabilities in the conventional sense (no injected malicious code, no exploited misconfiguration) — they were the kind of operational correctness bugs that a first real deployment to a fresh AWS account reliably surfaces: region-specific assumptions, implicit IAM permission requirements a cloud provider's own documentation omits, and a health-check assumption that silently breaks under a specific (increasingly common, security-motivated) container-image choice. Finding and fixing all eight before the service was billed as “working” is itself evidence for this project's broader thesis (Section 15): verification has to be real and adversarial against actual system behavior, not just “the terraform apply command exited 0.”

11. Validation & Testing

Actually run in this repository, per the README: **14 unit tests** (`app/tests`) and **3 integration tests** (`tests/integration`), all passing; ESLint clean. Run with:

```
make install
make test # unit tests (app/tests) + integration tests (tests/integration)
make lint
```

The gate engine itself has its own independent fixture-based test suite, `tests/security/run-tests.sh`, with **7 of 7 fixture scenarios passing**: gate passes when all five categories are within policy; gate fails on a threshold violation; gate fails on any secret finding (zero-tolerance); gate fails closed on a missing report; fails closed on a malformed (non-JSON) report; fails closed on a scanner-crashed status; and fails closed on a tool-not-installed status. This is what CI actually runs on every change to `security/scripts/security-gate.sh`, independent of the manual intentional-vulnerability walkthrough in Section 14.

```
make security # runs every installed scanner, then the gate
make security-test # tests the gate's own decision logic against fixtures
```

12. Limitations

Reproduced verbatim from the README's Limitations section:

- No DAST, fuzzing, or formal penetration testing.
- No CloudTrail/AWS Config deployed (documented as the largest gap versus a production-grade deployment).
- No automatic Secrets Manager rotation (manual rotation steps documented in `docs/troubleshooting.md`).
- No image signing / SLSA provenance attestation.
- Single NAT gateway (cost-optimized for a demo/student account, not multi-AZ-redundant).
- ALB serves plain HTTP (no ACM certificate/custom domain provisioned, to avoid the setup cost of a verified domain for a demo).
- SBOM generation (Syft) is best-effort and currently informational, not a gate input.
- In-memory user store (`app/src/models/userStore.js`) — no persistence across process/task restarts, and unreliable across Vercel serverless cold starts specifically. A real deployment would back it with RDS/DynamoDB.
- No frontend/UI and no AI/ML component — this is a backend API plus its surrounding CI/CD and security tooling only; the project's engineering focus is deliberately the pipeline, not the application.

13. Future Improvements

Reproduced verbatim from the README's Future Improvements section:

- Wire SBOM diffing into the gate (block on a new, unreviewed component appearing).
- Image signing (cosign/Sigstore) and SLSA provenance attestations, verified before ECS deploy.
- CloudTrail + AWS Config for full account-level audit trail.
- Automatic Secrets Manager rotation via a Lambda rotation function.
- Multi-AZ NAT gateways and an HTTPS listener with ACM for a production-grade (not just demo-grade) deployment.
- Rootless/Kaniko-style container builds to remove CodeBuild's `privileged_mode` requirement.
- Actually execute the `docs/research-notes.md` methodology against real pipeline runs and report measured DBR/VDR/FPR/MTTD/MTTR figures.

14. Local Setup & Reproduction Instructions

14.1 Prerequisites

An AWS account (a student/free-tier account is sufficient), AWS CLI v2 configured with credentials that can create IAM roles, VPCs, ECS/ECR/CodePipeline/CodeBuild resources, KMS keys, Secrets Manager secrets, and S3 buckets; Terraform $\geq 1.5.0$; Docker (for local builds/testing; CodeBuild builds images in AWS itself); a GitHub account and a fork/clone of this repository; Node.js ≥ 20 .

14.2 GitHub setup

- Push this repository to your own GitHub account/org.
- In the AWS Console: Developer Tools → Settings → Connections → Create connection, provider GitHub. Complete the OAuth handshake in the browser prompt — this is the one step Terraform cannot do for you, because it requires interactive GitHub authorization.
- Copy the resulting connection's ARN (`arn:aws:codeconnections:<region>:<account-id>:connection/<id>`) into `terraform.tfvars` as `codestar_connection_arn`.
- (Recommended) add a branch protection rule for main requiring the security-gate GitHub Actions status check to pass before merging.

14.3 Terraform state backend (one-time, per AWS account)

```
aws s3api create-bucket \
  --bucket <your-unique-tfstate-bucket-name> \
  --region us-east-1

aws s3api put-bucket-versioning \
  --bucket <your-unique-tfstate-bucket-name> \
  --versioning-configuration Status=Enabled

aws s3api put-bucket-encryption \
  --bucket <your-unique-tfstate-bucket-name> \
  --server-side-encryption-configuration \
  '{"Rules":[{"ApplyServerSideEncryptionByDefault":{"SSEAlgorithm":"AES256"}}]}'

aws dynamodb create-table \
  --table-name <your-unique-tf-lock-table-name> \
  --attribute-definitions AttributeName=LockID,AttributeType=S \
  --key-schema AttributeName=LockID,KeyType=HASH \
  --billing-mode PAY_PER_REQUEST
```

14.4 Terraform deployment

```
cd terraform/environments/dev

cp backend.tf.example backend.tf
# edit backend.tf: set bucket, dynamodb_table, region

cp terraform.tfvars.example terraform.tfvars
# edit terraform.tfvars: set github_owner, github_repo, codestar_connection_arn,
# and optionally alarm_notification_email

terraform init
terraform plan -var-file=terraform.tfvars
terraform apply -var-file=terraform.tfvars
```

terraform apply provisions, in order: the KMS key and Secrets Manager/SSM entries, the VPC/subnets/security groups, the ECR repository, the S3 pipeline-artifact bucket, all four IAM roles, the ECS cluster/ALB/service (seeded with a placeholder image tag — the first real image arrives via the pipeline), the CloudWatch log groups/alarms, the CodeBuild project, and finally the CodePipeline itself.

```
terraform output alb_dns_name # -> http://<dns-name>/health once a real image has deployed
terraform output ecr_repository_url
terraform output codepipeline_name

aws codepipeline start-pipeline-execution --name $(terraform output -raw codepipeline_name)
aws codepipeline get-pipeline-state --name $(terraform output -raw codepipeline_name)
```

14.5 Cost notes for a student/free-tier AWS account

- ECS Fargate task: 0.25 vCPU / 512 MB, 1 task — a few dollars/month if left running continuously; stop the ECS service (desired_count=0) or destroy the stack when not actively demoing it.
- One NAT gateway (not one per AZ) — NAT gateways bill hourly plus data processing; this is the single largest recurring cost in this stack.
- ECR storage is billed per GB; the lifecycle policy caps retained image history.
- CodeBuild BUILD_GENERAL1_SMALL is billed per build-minute; a full gated build typically runs 10-20 minutes depending on scanner install time.
- CloudWatch Logs/alarms and Secrets Manager (one secret) are a few cents a month at this scale.
- No ACM certificate / Route 53 hosted zone is provisioned (plain HTTP on port 80), avoiding the cost and friction of a verified custom domain for a demo deployment.

14.6 Local development

```
cd app
cp ../env.example .env # edit JWT_SECRET etc. for local use
npm install
npm run dev # starts on http://localhost:3000
curl http://localhost:3000/health
```

14.7 terraform destroy cleanup

```
cd terraform/environments/dev
terraform destroy -var-file=terraform.tfvars
```

This tears down everything Terraform created, except: the S3 tfstate bucket and DynamoDB lock table (created manually, delete separately); the CodeStar Connection (created manually via the AWS Console); and any images left in ECR if `force_delete` was not set (this repo's `aws_ecr_repository` does not set it, so `destroy` will fail if images remain — delete them first or add `force_delete = true` for a demo/dev environment).

15. Research Component Summary

This project doubles as a research artifact on whether automated, fail-closed enforcement changes deployment outcomes compared to advisory scanning. The full academic treatment, with a literature framing appropriate for an IEEE-format paper, is a separate document: `docs/research-paper-ieee.pdf`. This section summarizes only the measured-vs-proposed distinction, preserved exactly as `docs/research-notes.md` states it.

Three testable hypotheses (H1–H3) are defined, with exact metrics (Deployment Block Rate, Vulnerability Detection Rate, False Positive Rate, Mean Time To Detect, Mean Time To Remediate, Scan Overhead, Pipeline Execution Time, Fail-Closed Trigger Rate) and a methodology comparing a Gated pipeline configuration against an Advisory one against a corpus of clean and deliberately-vulnerable commits. **No specific DBR, VDR, FPR, MTTD, MTTR, or timing figure is stated anywhere in this repository as a measured result** — any such number would require actually executing that methodology against real pipeline runs, which has not been done.

What *has* actually happened: the infrastructure this methodology targets has been deployed to a real AWS account, and the CI/CD pipeline has run to completion (Source → Build → all five gate categories → Deploy) multiple times against real commits. This produced one piece of genuine empirical data — the eight-incident production debugging log in Section 10 — which `research-notes.md` is explicit is a **qualitative operational case study**, not a run of the DBR/VDR/FPR/MTTD/MTTR methodology. It says nothing about detection or enforcement rates for injected vulnerabilities and should not be read as such. What it does support is the more modest claim that the gate and infrastructure were exercised against real, unscripted AWS behavior — region-specific defaults, undocumented IAM permission requirements, a distroless base image's `$PATH` behavior — rather than only a `terraform plan` and a local test suite. The controlled-corpus trial (deliberately injected vulnerabilities, Gated vs. Advisory configurations, repeated runs) remains future work, exactly as described in `research-notes.md`.

What the logical-correctness test suite already establishes, separately from any measured claim: `tests/security/run-tests.sh` verifies, as a matter of code correctness (not a statistical claim), that the gate passes when all five categories are within policy, fails on a threshold violation, fails on any secret finding, and fails closed on a missing, malformed, or crashed-scanner report — establishing that the mechanism behaves as designed, which is a distinct claim from H1–H3 about its effect across many real pipeline executions over time.

16. Live Demo Access

Two separate, genuinely running deployments exist. They should not be confused with each other:

	AWS production pipeline	Vercel API demo
What it is	The actual system this guide describes: GitHub → CodePipeline → CodeBuild → 5 security gates → ECR → ECS Fargate → ALB	A supplementary, publicly reachable copy of just the Express API, for quick poking without needing AWS access

	AWS production pipeline	Vercel API demo
URL	secure-cicd-dev-alb-907205721. ap-south-1.elb.amazonaws.com	secure-aws-cicd-demo-api. vercel.app
Try it	curl http://secure-cicd-dev-alb-907205721. ap-south-1.elb.amazonaws.com/health	curl https://secure-aws-cicd-demo-api. vercel.app/health
Runs the security gate?	Yes — every deploy	No — Vercel's build does not run Semgrep/Trivy/Checkov/security-gate.sh
Persistence	ECS task, in-memory user store (resets on task restart)	Serverless function, in-memory user store (resets on every cold start — expect registered users to disappear between requests)

Table 8. Verbatim from the README's Live demo section.

The ALB target is a **student/demo AWS account kept running only while this project is being actively evaluated** — plain HTTP (no ACM certificate/custom domain provisioned), single NAT gateway, smallest Fargate task size. It is not a cost-optimized-for-uptime production service and may be torn down (terraform destroy) after evaluation to stop AWS billing; the Vercel demo is the more durable of the two public endpoints. Both were verified reachable from a clean, unauthenticated request as of the last update to the README.

16.1 Demo walkthrough

- curl the AWS ALB /health endpoint above — that response came from a real ECS Fargate task that only exists because a CodeBuild run passed all five security gates.
- Push a clean commit to main — watch the pipeline go Source → Build (gate PASS) → Deploy in the CodePipeline console.
- Run the intentional-vulnerability demo locally or push one of its scenarios to a branch — watch the gate FAIL and the Deploy stage never run.
- Revert and push again — watch it PASS.
- Read docs/production-incidents.md for what actually broke (and how it was diagnosed) getting this pipeline to a genuinely healthy state the first time.

16.2 Intentional-vulnerability demonstration (summary)

docs/deployment.md section 10 walks through tripping each scanner on purpose, in isolation, and watching the gate block the build, then reverting to show it pass again:

- **Dependency scanner:** install a package with a long-published CVE (minimist@0.0.8) and re-run the gate — expect FAIL.
- **Secret scanner:** add a clearly-marked fake credential file and re-run — expect FAIL (zero-tolerance).
- **SAST scanner:** add an isolated, never-imported file with a command-injection pattern and re-run — expect FAIL.
- **IaC scanner:** append a deliberately open (0.0.0.0/0 on port 22) security group resource and re-run — expect FAIL.
- **Container scanner:** scan an intentionally old, unpatched public image (node:18.0.0-alpine) instead of the project's own build — expect FAIL.

- **Fail-closed behavior directly:** delete a report file and re-run the gate with no scanner involved — expect FAIL with a “report file missing ... (fail-closed)” message.

This maps 1:1 to the automated version of the same checks in `tests/security/run-tests.sh` (make `security-test`), which is what CI actually runs on every change to the gate — the manual walkthrough is for a live demo audience, the automated one is what keeps the gate correct over time.

17. Appendix: Repository Structure Reference

Reproduced from the README's Repository structure section:

```
app/ Node.js/Express demo API + Dockerfile + Jest tests
security/
  policy/ security-policy.yaml -- configurable thresholds
  scripts/ one wrapper script per scanner + security-gate.sh
  reports/ generated scan output (gitignored, .gitkeep only)
terraform/
  modules/ networking, iam, ecr, ecs, codebuild, codepipeline,
  cloudwatch, security
  environments/dev/ concrete dev deployment wiring the root module
ci/
  buildspec.yml the real CodeBuild pipeline definition
  scripts/ ecr-login.sh, push-image.sh, generate-sbom.sh
tests/
  security/ tests for the gate engine itself (fixture-based)
  integration/ end-to-end API test
docs/ architecture, security model, deployment, threat
  model, troubleshooting, research notes
diagrams/ standalone Mermaid sources
.github/workflows/ security-gate.yml, terraform-validate.yml
```

End of guide. This document was generated from the repository's current documentation (README.md and docs/) and source configuration (terraform/, ci/buildspec.yml, security/policy/security-policy.yaml) as of the last update to those files; no technical claim in this document is stated without a corresponding source in the repository.